

목차

List<Project> projects

{

1. Fantasy Hero TD, 프로젝트 개요, 구현 내용, 개발 중 문제 상황과 해결
2. 젤다의 전설 기반 창작게임 프로젝트 개요, 구현 내용 – 엔진, 구현내용 – 클라이언트, 개발 중 문제 상황과 해결
3. 슈퍼 페이퍼 마리오 모작 프로젝트 개요, 구현 내용, 개발 중 문제 상황과 해결
4. 리그 오브 레전드 모작 구현 내용, 개발 중 문제 상황과 해결

};

 노션 바로가기 → https://chlorinated-finch-e95.notion.site/1040f8ae9ea2804b9591cb6d7e7f300f?source=copy_link

 GitHub 저장소 바로가기 → <https://github.com/dk60116>

Project Fantasy Hero TD

사용 기술: **Unity, C#, Google Fierebase SDK**

제작 인원: 개인 개발

[프로젝트 개요](#)

[구현 내용](#)

[개발 중 문제 상황과 해결](#)



📺 영상 바로가기 → <https://youtu.be/EcHQHZTO9To?si=O16ty4umqQ7JFnil>

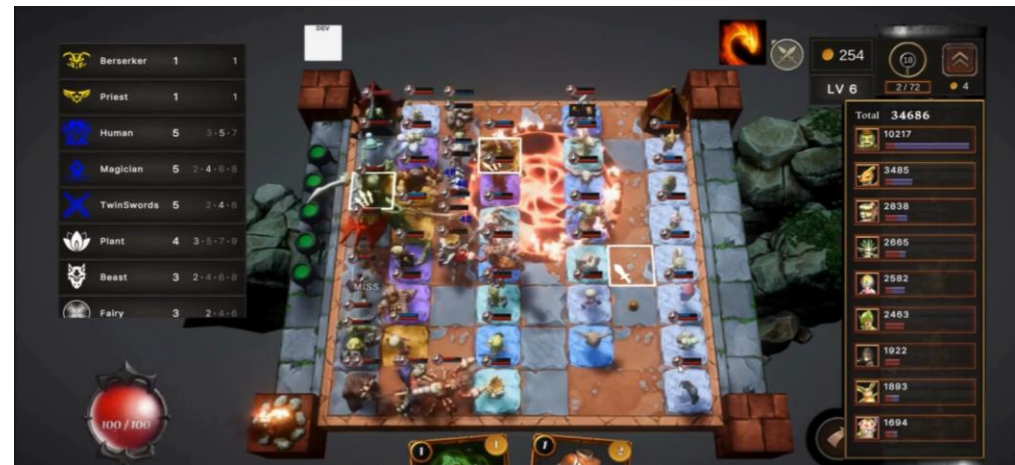
🔗 GitHub 저장소 바로가기 → [FHTD-Project/FHTDProject/Assets/Scripts at main · dk60116/FHTD-Project](https://github.com/dk60116/FHTD-Project)

프로젝트 개요

게임 설명

TFT(롤토체스) + 타워디펜스 (모바일)

보유한 재화(골드)로 유닛을 구매해 배치 한 뒤 시작 버튼을 누르면
몰려오는 적들을 제본진과 적 기지의 위치를 자유롭게 배치할 수 있음. 위치에 따라 A* 알고리즘으로 경로가 자동 생성되며, 유닛이나 장애물의 위치에 따라 경로를 다시 계산함.
같은 레벨의 동일한 유닛을 3개 모으면 레벨업을 하며, 9개를 모아 최대 3레벨까지 성장 가능.
최대배치 할 수 있는 유닛들(영웅)은 모두 스킬을 가지고 있으며, 방어할 때와 공격할 때의 스킬이 각각 존재함.
유닛은 종족, 직업이 있으며, 같은 종족이나 직업의 유닛들을 N개 이상 배치하면 추가적인 버프나 시너지를 받을 수 있음
CCG 요소를 가미해 보유한 카드들로 아군 유닛에 버프를 주거나 적에게 타격이나 디버프를 줄 수 있음.
몰려오는 적들을 제거해 아군 기지에 도착하지 못하게 하여 방어하는 게임



구현 내용

1. Google Firebase SDK를 사용한 유저 데이터 저장
2. CSV 기반의 Data Table로 게임 데이터 관리
3. TCG 스타일의 카드 수집, 덱 메이킹 시스템
4. 50종 이상의 다양한 캐릭터, 스킬 구현
5. 오토체스 식의 랭크별 유닛 상점 시스템 구현
6. A* 알고리즘으로 구현한 적 유닛 이동 경로 시스템
7. 카드 스킬 구현



AI																					
ID	Name	Rank	MoveSpeed	MaxHP	MaxMP	StartMP	AD	Damag	APower	AtkSpeed	AtkRange	ADDef	APDef	Tribe1	Class1	Class2	Projectile	AtkDelay	ProjSpeed	DirectAtk	Air
101000	Belber	Common	0.36	145	80	50	22	60	0.37	1	20	30	Plant	Fighter			0.35			FALSE	
101001	Cabino	Common	0.4	140	50	10	25	80	0.5	1.3	15	30	Plant	BigHorns			0.35			FALSE	
101002	Dinko	Common	0.4	145	60	20	16	100	0.55	3	15	30	Dragon	Flying		1	0.3	10		TRUE	
101003	Azzang	Common	0.3	200	100	70	35	100	0.45	1.2	46	30	Beast	Insect			0.25			TRUE	
101004	Moya	Common	0.4	180	120	20	29	65	0.5	1.2	25	30	Dragon	Fighter			0.8			TRUE	
101005	FingerElf	Common	0.45	160	0	0	28	70	0.65	1.3	10	30	Elemental	Fighter			0.25			FALSE	
101006	Wacool	Common	0.4	180	50	30	33	90	0.5	1.1	25	30	Undead	Natural			0.3			FALSE	
101007	Soskereu	Common	0.4	150	50	20	14	100	0.45	3.5	20	30	Undead	Ranger		1	0.1	10		TRUE	
101008	Keurst	Common	0.35	165	60	50	32	80	0.45	1.25	32	30	Beast	Armoric			0.4			FALSE	
101009	Witch	Common	0.4	137	70	30	18	100	0.4	3	15	30	Human	Magician		1	0.2	12		FALSE	
101010	Assasin	Common	0.55	152	30	0	15	45	1.5	1	18	30	Human	TwinSwords			0.1	8		FALSE	
101011	Mocomoc	Common	0.5	140	60	40	22	90	0.52	2	15	30	Fairy	Magician		1	0.25	15		FALSE	
101012	Godongl	Common	0.35	190	70	40	20	65	0.42	1	28	30	Fairy	Armoric			0.3			FALSE	
101013	Peullamor	Common	0.4	140	80	20	85	0.5	1.8	27	30	Plant	Insect		1	0.3	15		FALSE		
102000	Hunter	UnComm	0.5	180	50	20	12	50	0.65	4	15	30	Human	Ranger		1	0.3	25		FALSE	
102001	Dragon	UnComm	0.55	186	110	70	14	85	0.55	2.5	20	30	Dragon	Natural		1	0.2	10		TRUE	
102002	Bellial	UnComm	0.5	295	80	40	28	80	0.6	2.7	10	30	Elemental	Natural		1	0.2	15		FALSE	
102003	Okin	UnComm	0.5	310	0	0	20	40	0.22	1.2	34	30	Undead	Fighter			0.5			FALSE	
102004	Guroguro	UnComm	0.4	375	60	20	20	65	0.6	1	45	30	Elemental	Armoric			0.35			TRUE	
102005	Preditor	UnComm	0.45	295	80	30	20	90	0.5	2	20	30	Undead	Magician		1	0.25	15		FALSE	
102006	Basilisk	UnComm	0.35	320	80	40	20	50	0.4	1	40	30	Beast	Armoric			0.1			FALSE	
102007	Cablo	UnComm	0.4	330	110	20	20	40	0.55	1.2	35	30	Beast	BigHorns			0.6			FALSE	
102008	Babarian	UnComm	0.4	295	50	20	20	55	0.55	1.2	40	30	Human	Armoric			0.15			FALSE	
102009	Icefairy	UnComm	0.55	260	70	30	8	85	0.6	2.5	20	30	Fairy	Flying		1	0.25	10		TRUE	
102010	Kare	UnComm	0.5	320					0.5	1.1	35	30	Fairy	Fighter			0.4			FALSE	
102011	Sporina	UnComm	0.5	275					0.55	2	26	30	Plant	Natural		1	0.3	10		FALSE	
102012	Hogs	UnComm	0.4	320	90	70	20	60	0.5	1	32	30	Plant	Armoric			0.5			FALSE	
102013	Singilee	UnComm	0.45	280					0.6	2	15	30	Plant	Natural		1	0.2	10		FALSE	
103000	Syaonil	Rare	0.55	390	70	20	32	70	0.48	3.5	40	30	Undead	Flying	Swordsmen		0.25	50		TRUE	
103001	Magemon	Rare	0.5	365					0.4	2	40	30	Dragonoid	Swordsmen			0.1		1	FALSE	
103002	Scollwarik	Rare	0.4	590	100	40	20	85	0.7	1.8	55	30	Guardian	Insect			0.4			FALSE	



문제1: 데이터가 방대해지는 문제

50종에 달하는 유닛과 많은 카드 종류 때문에 관리해야 할 데이터 수가 너무 많아 졌음

기존 방식

스탯 관리가 프리팹 단위로 하드코딩 되어 있어 데이터 수정 확장 시 어려움이 있음.

해결법

CSV(Text Asset) 기반의 데이터 관리 파이프라인 구축

성과

콘텐츠 추가/밸런싱을 데이터 수정만으로 처리 가능해져 개발 생산성 향상.

파이프라인 통합으로 데이터 로딩 흐름이 단순화되고, 유지보수성이 높아짐.

깃허브 코드 보기

DBReader.cs <https://github.com/dk60116/FHTD-Scripts/blob/main/Scripts/Unit/Unit.cs>

ID	Name	Rank	MoveSpeed	Health	MaxHp	MaxMp	Attack	Defense	Agility	Stamina	Intelligence	Element	Class	Projectile	Armor	Progression	IsElite
101000	Barber	Common	0.35	140	80	30	22	40	0.37	1	20	30	Plane	Fighter	0.35		FALSE
101001	Calder	Common	0.4	140	30	10	25	80	0.3	1.3	15	30	Plane	Big Horn	0.35		FALSE
101002	Drao	Common	0.4	140	40	20	16	100	0.35	3	15	30	Dragon	Flying	1	0.3	10
101003	Azany	Common	0.3	200	100	70	35	100	0.45	1.2	46	30	Beast	Insect	0.25		TRUE
101004	Kopa	Common	0.4	180	120	100	29	65	0.5	1.2	25	30	Dragon	Fighter	0.8		TRUE
101005	Fingelle	Common	0.45	180	0	0	28	70	0.65	1.3	10	30	Elemental	Fighter	0.25		FALSE
101006	Nazul	Common	0.4	180	30	30	31	90	0.5	1.5	25	30	Undead	Natural	0.3		FALSE
101007	Sokenu	Common	0.4	150	50	20	14	100	0.45	1.5	20	30	Undead	Ranger	1	0.1	10
101008	Nazul	Common	0.35	160	40	30	32	80	0.45	1.25	52	30	Beast	Armored	0.4		FALSE
101009	Witch	Common	0.4	137	70	30	18	100	0.4	3	15	30	Human	Magician	1	0.2	12
101010	Assault	Common	0.35	150	30	0	15	45	1.5	1	18	30	Human	Tank/Support	0.1		FALSE
101011	Moonmo	Common	0.5	140	80	40	22	90	0.52	2	15	30	Fairy	Magician	1	0.25	15
101012	Goading	Common	0.35	140	70	40	20	65	0.42	1	28	30	Fairy	Armored	0.3		FALSE
101013	Paulaner	Common	0.4	140	40	20	85	0.5	1.8	27	30	Plane	Invest	1	0.3	15	
102000	Hunter	UNCComm	0.5	180	50	20	12	50	0.65	4	15	30	Human	Ranger	1	0.3	25
102001	Dragon	UNCComm	0.55	188	110	70	14	85	0.55	2.5	20	30	Dragon	Natural	1	0.2	10
102002	Infant	UNCComm	0.5	295	80	40	28	80	0.6	2.7	10	30	Elemental	Natural	1	0.2	15
102003	Clan	UNCComm	0.5	375	0	0	20	40	0.22	1.2	34	30	Undead	Fighter	0.5		FALSE
102004	Gungary	UNCComm	0.4	375	60	20	20	65	0.6	1	45	30	Elemental	Armored	0.35		TRUE
102005	Proditor	UNCComm	0.45	295	80	30	20	90	0.5	2	20	30	Undead	Magician	1	0.25	15
102006	Baillak	UNCComm	0.25	320	80	40	20	50	0.4	1	40	30	Beast	Armored	0.6		FALSE
102007	Galio	UNCComm	0.4	330	110	20	20	40	0.55	1.2	30	30	Beast	Big Horn	0.6		FALSE
102008	Subarian	UNCComm	0.4	295	50	20	20	55	0.55	1.2	40	30	Human	Armored	0.15		FALSE
102009	Infatry	UNCComm	0.55	260	70	30	8	85	0.6	2.5	20	30	Fairy	Flying	1	0.25	10
102010	Kare	UNCComm	0.5	320	0	0	0	0.5	1.5	30	30	Fairy	Fighter	0.4		FALSE	
102011	Spodro	UNCComm	0.5	275	0	0	0	0.55	2	28	30	Plane	Natural	1	0.3	10	
102012	Hoop	UNCComm	0.4	320	90	70	20	60	0.5	1	32	30	Plane	Armored	0.5		FALSE
102013	Single	UNCComm	0.45	280	20	20	20	0.6	2	15	30	Plane	Natural	1	0.2	10	
103000	Synard	Rare	0.55	390	70	20	12	70	0.48	3.5	40	30	Undead	Flying	0.25		10
103001	Magemon	Rare	0.5	385	0	0	0	0.4	2	40	30	Dragonoid	Swindman	0.1		FALSE	
103002	Schwartz	Rare	0.4	590	100	40	20	85	0.7	1.8	55	30	Guardian	Insect	0.4		FALSE

E Rank	Common
F Org Move Speed	0.35
F Crt Move Speed	0.35
I Max Hp	165
I Crt Hp	165
I Max Mp	60
I Crt Mp	50
I Shield	0
I Org Start Mp	50
I Crt Start Mp	0
Increase Mp_Hit	0
I Org Attack	32
I Crt Attack	32
I Add Attack	0
I Org Abil Power	80
I Crt Abil Power	80
I Add Power	0
F Org Atk Speed	0.45
F Crt Atk Speed	0.45
F Org Cri Rate	10

문제2: 다중 버프 중첩 관리

같은 효과가 여러 번 중첩되면 지속시간과 효과 수치를 어떻게 관리할 것인가?

1. 기존 남은 시간은 유지하고 새로운 효과 유지시간 더하기

지속시간이 너무 길어 짐

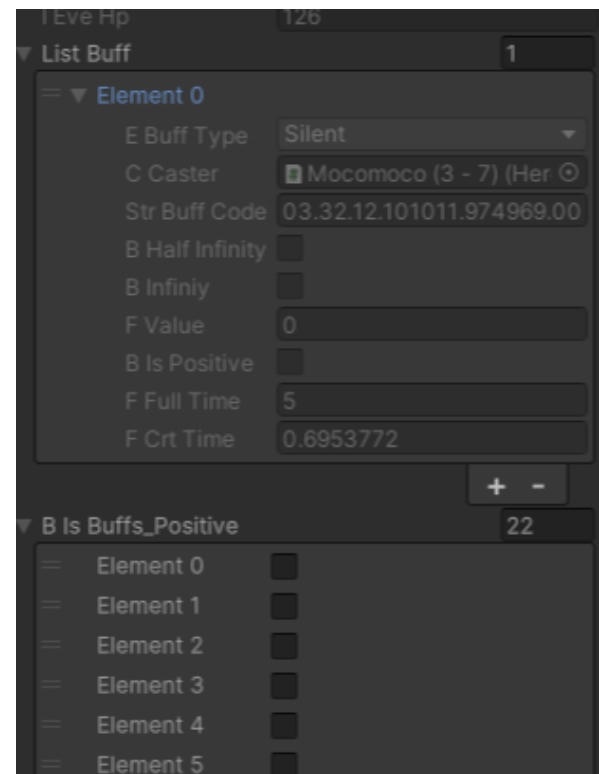
2. 기존 버프와 새 버프를 비교 후 짧은 버프를 없애고 새로운 버프를 적용 하기

지속시간 문제는 해결이 되지만 효과 수치 중첩에 대한 처리가 안됨

3. 버프를 List화, 매 프레임 순회하며 버프가 변화시키는 스탯을 적용 후 남은 시간을 확인해 순차적으로 제거하는 방식

개별 버프 인스턴스 기반으로 버프 지속시간 및 중첩 관리에 용이하고

+ 버프 관리의 안정성 확보



깃허브 코드 보기

Unit.cs: <https://github.com/dk60116/FHTD-Scripts/blob/main/Scripts/Unit/Unit.cs>

Project 젤다의 전설 기반 창작게임

사용 기술: **DirectX11, C++**

제작 인원: 개인 개발

프로젝트 개요

구현 내용 - 엔진

구현내용 - 클라이언트

개발 중 문제 상황과 해결



▶ 게임 영상 바로가기 → https://youtu.be/fczruTgZKws?si=zS343slQ_VDzvbbg

▶ 엔진 영상 바로가기 → <https://youtu.be/OL0d0pIPi6o?si=T9rv1SPawYsdYbHY>

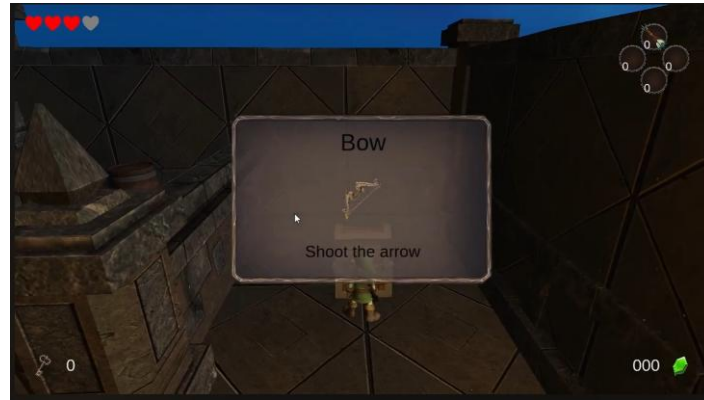
🔗 GitHub 저장소 바로가기 → https://github.com/dk60116/TK_Repository/tree/Temp/DX11Engine

프로젝트 개요

게임 설명

닌텐도의 대표작 젤다의 전설 시리즈를 나만의 스타일로 모작.

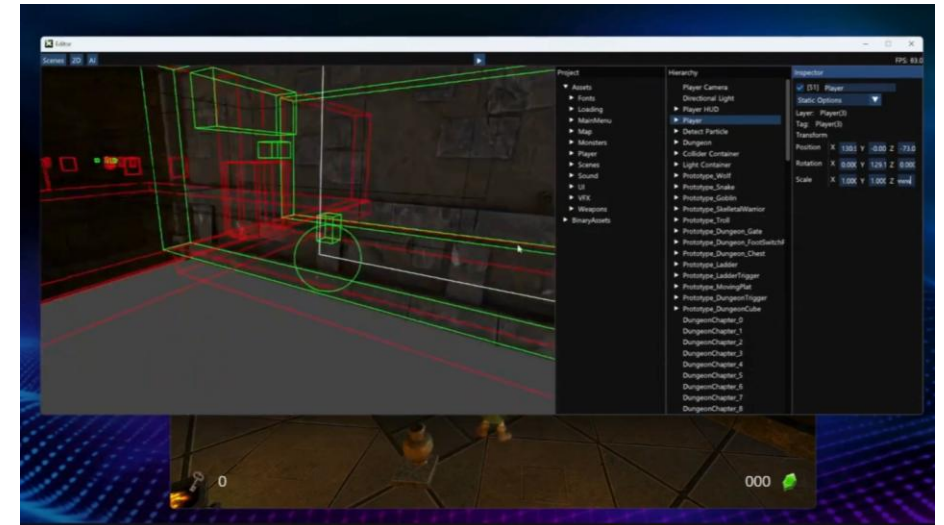
던전을 탐험하며 검과 활, 화살로 몬스터를 물리치고 스위치, 트리거, 플랫폼 등의 퍼즐을 풀며 열쇠를 획득하여 최종보스인 드래곤을 물리쳐 클리어 하는 기본에 충실한 게임.



구현 내용 - 엔진

유니티 엔진 프레임 워크를 모방한 게임엔진 시스템

1. 스왑체인 방식의 Game, Editor, Debug 디스플레이 분리
2. Hierachy, Project 패널
3. Unity 스타일의 Scene - Game Obejct 라이프 사이클
4. 멀티스레딩으로 로딩 시간 중 클라이언트 동작
5. Defferd Shading
6. Vector2, Vector3, Vector4, Quaternion, Color 등의 구조체 재정의
7. Camera, Light, MeshRenderer, Animator, Collider 등의 Monobehaviour기반 컴포넌트 시스템 구현
8. Alpha Blending, Mesh Instancing, GPU Instancing Material, HLSL Shader 등 다중 광원 처리 등의 렌더 파이프라인
9. OBB Box, Sphere 두가지의 Collider, 중력과 충돌처리 등의 기본적인 물리엔진 구현
10. RectTransform 기반으로 구동되는 UI시스템
11. 파티클 및 이펙트 시스템
12. Fmod SDK를 사용한 64비트 사운드 시스템



구현 내용 - 클라이언트

1. 주인공 캐릭터의 조작, 애니메이션 제어, 카메라 제어
2. 고블린, 뱀, 늑대, 스켈레탈, 트롤, 드래곤(보스) 등의 몬스터들과 FSM을 사용한 AI
3. 검 애니메이션, 이펙트, 활과 화살 이펙트
4. 스위치 및 트리거, 사다리, 문, 상자, 큐브, 이동 플랫폼 등의 특수 오브젝트
5. 맵과 콜라이더 배치
6. 사운드 이펙트

문제: 콜라이더 최적화

문제 상황: 수백개의 콜라이더 충돌처리를 매 프레임 처리하기 위한 최적화

OBB, Box to Sphere 등이 맵 전체에 분포되어 매 프레임 충돌검사를 하기엔 연산을 너무 많이해 정상적인 프레임으로 동작할 수 없음.

해결방법1: 유니티 프레임워크처럼 FixedUpdate를 Update(매 프레임)와 같은 주기로 동작하지 않고 특정 시간마다 주기적으로 호출 (ex 0.02초 마다 동작)

해결방법2: Physics Layer시스템을 만들어 특정 레이어에 해당하는 콜라이더 끼리는 연산하지 않도록 제어

해결방법3: 클라이언트 단계 - 맵 전체의 구역을 나누어 해당 구역에 들어가야 콜라이더가 활성화 되도록 구현. 콜라이더가 없으면 몬스터 등의 물리효과를 받는 물체가 떨어지기 때문에 입장과 동시에 해당 구역의 오브젝트들도 활성화 했음.

성과: 20 Frame 미만의 프레임 -> 100프레임 이상의 원활한 동작

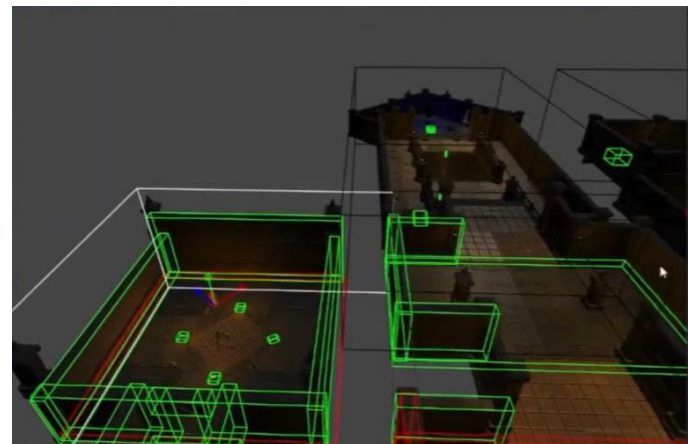
깃허브 코드 보기

Unit.cs: [https://github.com/dk60116/TK_Repository/blob/49049f9f040f28](https://github.com/dk60116/TK_Repository/blob/49049f9f040f287a197bb70c41dddf996ea8737e/DX11Engine/Engine/Code/CollisionManager.cpp)

[7a197bb70c41dddf996ea8737e/DX11Engine/Engine/Code/CollisionManager.cpp](https://github.com/dk60116/TK_Repository/blob/49049f9f040f287a197bb70c41dddf996ea8737e/DX11Engine/Engine/Code/CollisionManager.cpp)

Scene.cs: [https://github.com/dk60116/TK_Repository/blob/49049f9f040f287a197bb](https://github.com/dk60116/TK_Repository/blob/49049f9f040f287a197bb70c41dddf996ea8737e/DX11Engine/Engine/Code/Scene.cpp)

[70c41dddf996ea8737e/DX11Engine/Engine/Code/Scene.cpp](https://github.com/dk60116/TK_Repository/blob/49049f9f040f287a197bb70c41dddf996ea8737e/DX11Engine/Engine/Code/Scene.cpp)



문제: 타일맵으로 된 수천개의 오브젝트

에셋이 타일맵을 여러 개 소환하는 방식으로 되어있어 소환해야할 Mesh Renderer가 너무 많아짐.

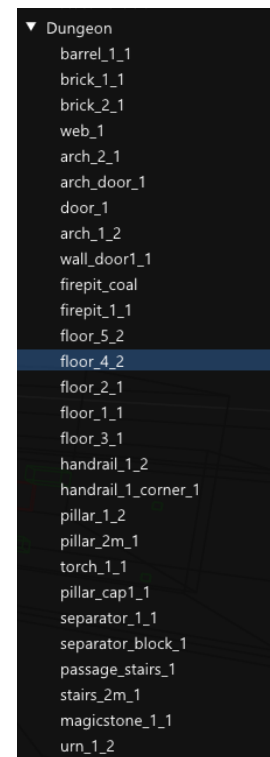
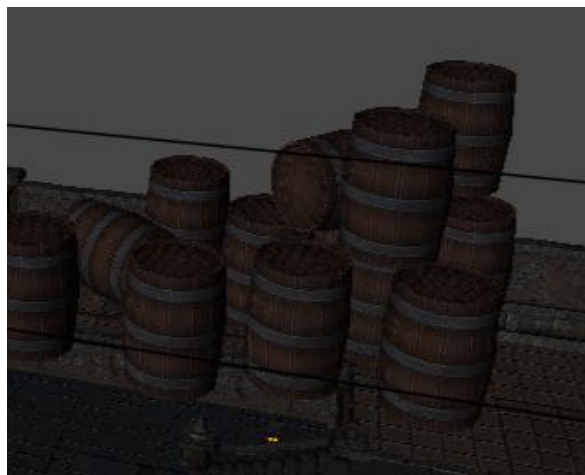
해결법

Mesh Instancing 기법으로 모양은 같고 Transform 정보가 다른 같은 모양의 오브젝트를 MeshBuffer 하나로 합쳐 드로우콜 감소

성과

소환되어 관리해야 할 게임 오브젝트 2500개 이상 -> 30개로 감소

& 드로우콜 감소로 프레임 대폭 향상



문제: 파티클 이펙트 셰이더

메시를 수십~수백개 소환하여 움직이는 파티클의 Drawcall을 줄여야 한다.

해결법

파티클 시스템 컴포넌트를 만들고 메시는 MeshInstancing으로 소환,
초기 위치, 끝 위치, 초기 색, 끝프레임 색 방향과 속도 등을 모두 GPU에 넘겨
셰이더 단계에서 계산하게 하여 병렬처리

깃허브 코드 보기

ParticleSystemcs: https://github.com/dk60116/TK_Repository/blob/Temp/DX11Engine/Engine/Code/ParticleSystem.cpp

Particle.hlsl: https://github.com/dk60116/TK_Repository/blob/Temp/DX11Engine/Engine/EngineResources/Shader/Particle.hlsl



Project 슈퍼 페이퍼 마리오 모작

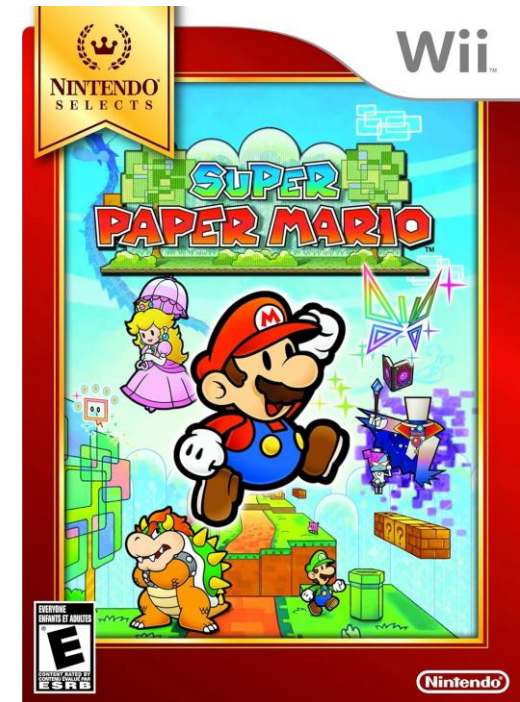
사용 기술: **DirectX9, C++**
제작 인원: 클라이언트 4인

프로젝트 개요

구현 내용

개발 중 문제 상황과 해결

▶ 영상 바로가기 → <https://youtu.be/qJ5AlzZo62U?si=ZUCepwdM87mLSeak>
▶ GitHub 저장소 바로가기 → <https://github.com/dk60116/SuperPaperMario.git>



프로젝트 개요

게임 설명

2D-3D를 오가는 액션의 플랫폼머. 닌텐도의 대표게임 마리오 IP로 한 작품

2D ↔ 3D 전환(차원이동술): 버튼 한 번으로 2차원과 3차원을 오가며 퍼즐과 전투를 해결합니다.
원작의 1스테이지와 2스테이지를 구현했으며 마지막 보스를 처치해 게임을 클리어합니다.



구현 내용 (✓는 나의 작업내용)

1. 2D 캐릭터의 신체 부품별 애니메이션 ✓
2. 플레이어 파티클 ✓
3. 백버퍼를 복사해 뒤집는 화면 전환 이펙트 ✓
4. 중력 가속도 적용으로 자연스러운 점프와 하강. ✓
5. 3D <-> 2D 체인지 기능
6. 맵툴
7. PhysicsX를 이용한 물리 충돌처리, 2D충돌 처리를 위한 커스텀 Collider 시스템
8. 이벤트 오브젝트
9. 몬스터 및 이펙트
10. 보스 몬스터와 패턴 ✓

문제: 2D 이미지의 파트 애니메이션 재생

캐릭터 하나가 머리 몸통, 팔, 다리 등 여러 부품으로 나뉘어 있어 애니메이션 구동이 복잡함.

해결법

Transform에서 부모-자식 객체 만들고 각 파트를 이어 붙인 뒤 리스트화,
DO-Tween과 유사한 선형보간하는 애니메이션 시스템을 제작,
이후 캐릭터를 뒤집어도 보이게 CCW컬링을 해제 했음.

성과

마리오와 보스의 파트 애니메이션이 자연스럽게 구동되었으며 이 시스템을 몬스터 담당 클라이언트에게 넘겨
몬스터 애니메이션도 안정적으로 완성.



문제: 2D – 3D를 오가는 충돌처리

문제점

3D와 2D 상태의 콜라이더가 완전히 달라 프로젝트 진행에 어려움이 있어 많은 회의를 통해 해결법 도출

해결

3D 모드에서는 PhysicsX SDK를 사용해 충돌 구현, 2D모드에선 3D때의 물리 상호작용을 모두 끄고 직접 제작한 AABB 콜라이더를 이용하여 구분하여 개발을 완료했음.

Project League of Legends 모작

프로젝트 개요

사용 기술: Unreal Engine, C++

구현 내용
개발 중 문제 상황과 해결

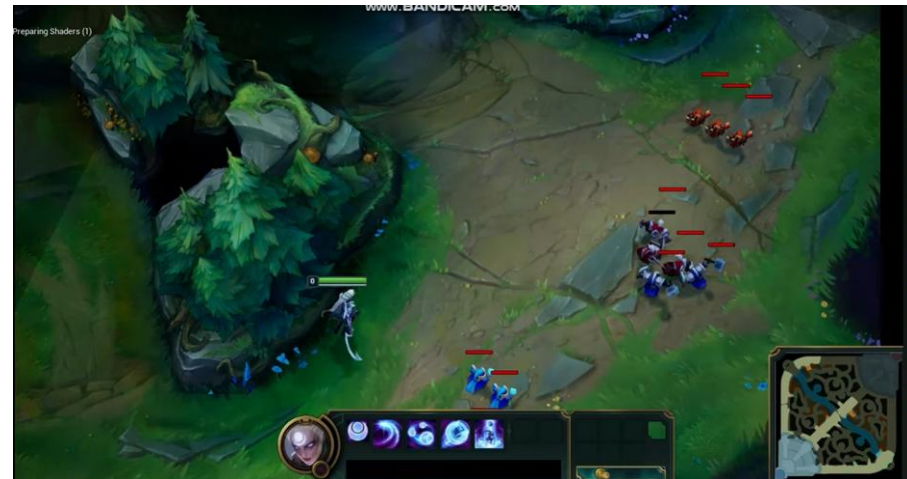


▶ 영상 바로가기 → <https://youtu.be/qWUQJPIMMrk?si=aIVB1uSgGCs2XNYV>

▶ GitHub 저장소 바로가기 → <https://github.com/dk60116/LeagueOfLegendsProject>

구현 내용

1. 플레이어 (다이애나) 캐릭터 제작, 애니메이션 관리
2. 미니언과 미니언 라인, AI BehaviourTree작성
3. 넥서스, 억제기, 타워
4. 전장의 안개(Fog of war) 시스템 구현 및 미니맵에 적용



문제: 전장의 안개 시스템 프레임 드랍

문제 상황 : 브레젠햄 알고리즘을 통해 시야를 판정해 1024 x 1024 이상의 해상도의 텍스처를 구워 맵에 렌더하는 Fog Of War 시스템을 구현했으나 하지만 미니언, 챔피언, 타워 등이 전부 시야를 가지고 있어 Observer가 많은 상태로 CPU에서 돌리면 에디터 기준 20Frame 미만으로 동작함.

해결방법: GPU 병렬 연산을 위한 Compute Shader 구현

성과: 에디터 기준 30FPS미만의 저성능 -> 120FPS이상으로 최적화



깃허브 코드 보기

FogGrid: https://github.com/dk60116/LeagueOfLegendsProject/blob/main/Source/ProjectL_5/Private/Map/FogGrid.cpp

FogOfWarCS.usf: <https://github.com/dk60116/LeagueOfLegendsProject/blob/main/Shaders/ComputeShader/FogOfWarCS.usf>